Análisis de Hardware y Portabilidad
Mi ESP32 y ESP32 facultad

Guillermo Casco Miranda

Este documento establece una comparativa técnica detallada entre el kit de desarrollo propio (Muse Lab nanoESP32-C6, referido de acá en adelante como "mi kit" o "mi ESP32") y la placa provista por la facultad (ESP-EDU v1.1, refererida de acá en adelante como "ESP32 facultad" aludiendo al MCU, kit, o placa, como si fueran lo mismo). A pesar de sus diferencias físicas y periféricos integrados, el análisis de los logs de arranque y la inspección visual confirman que ambas placas utilizan exactamente el mismo microcontrolador base: el ESP32-C6. Esto garantiza una altísima compatibilidad a nivel de arquitectura y código central.

Especificación / Característica	Mi ESP32: en kit de desarrollo Muse Lab nanoESP32-C6	ESP32 facultad: enkit de desarrollo DevKitC-1, montado en placa desarrollo ESP-EDU v1.1
Módulo Físico (SoC)	ESP32-C6-WROOM-1 (V 1.2)	ESP32-C6-DevKitC-1 / WROOM-1
Arquitectura y Núcleos	RISC-V 32-bit (RV32IMAC), Single-Core	RISC-V 32-bit (RV32IMAC), Single-Core
Velocidad del Reloj	160 MHz	160 MHz
Memoria RAM (SRAM)	512 KB (aprox. 475 KB libres para app)	512 KB (aprox. 475 KB libres para app)
Memoria Flash (Física)	8 MB (Identificada como chip genérico)	8 MB (Identificada como chip GigaDevice)
Revisión del Silicio	v0.2 (Revisión más reciente)	v0.0 (Primera revisión del chip)
Wi-Fi	Wi-Fi 6 (802.11 ax/b/g/n) 2.4 GHz	Wi-Fi 6 (802.11 ax/b/g/n) 2.4 GHz
Bluetooth	Bluetooth Low Energy (BLE 5.3)	Bluetooth Low Energy (BLE 5.3)
Zigbee / Thread / ESP-NOW	Sí (Nativo vía IEEE 802.15.4 y Wi-Fi)	Sí (Nativo vía IEEE 802.15.4 y Wi-Fi)
LoRaWAN	No nativo (requiere m ódulo externo)	No nativo (requiere m ódulo externo)
ADC / DAC Integrados	ADC 12-bit (7 canales). **No posee DAC.**	ADC 12-bit (7 canales). **No posee DAC.**
Periféricos Integrados	1x LED RGB (WS2812), Botones BOOT/RST	3 LEDs (LED_1 a GPIO11, LED_2 a GPIO10, LED_3 a GPIO5) 2 pulsadores (SWITCH_1 a GPIO4, y SWITCH_2 a GPIO15), Cargador Bater ía (MCP73831), Botones BOOT/RST

Arquitectura y Portabilidad de Código

Es imperativo aclarar que el ESP-EDU de la facultad monta el módulo ESP32-C6-WROOM-1.

Por lo tanto, mi kit Muse Lab y el kit de la facultad son gemelos a nivel de silicio: ambos son microcontroladores unicore basados en la arquitectura RISC-V.

El set de instrucciones (RV32IMAC), la gestión de FreeRTOS y las rutinas de interrupción funcionarán de manera idéntica en ambas placas sin necesidad de adaptaciones de bajo nivel.

> **Pines de Strapping (Comunes a ambas placas)**
>
> El comportamiento de arranque del ESP32-C6 está gobernado por niveles de voltaje en pines específicos al momento de energizar o presionar RESET. Se conectan cargas y sensores convenientemente para evitar conflictos al arranque y reseteto (GPIO8 con relevador, GPIO9 con sensor de gas, GPIO15 con tecla 2):
>
> - **GPIO8:** Controla los mensajes de ROM (1 = Imprime por consola, 0 = Silencioso).
> - **GPIO9:** Pin de Modo de Arranque (1 = Arranca código desde Memoria Flash, 0 = Entra en Modo Download/Flasheo). *Este pin está físicamente conectado al pulsador BOOT en mi kit Muse Lab.*
> - **GPIO15:** Señal JTAG y timing (Debe estar en 1 para operación normal).

Diferencias a nivel de Periféricos (Mapeo de GPIOs)

La placa Muse Lab expone los pines crudos de manera directa sobre el protoboard. En contraste, el esquemático del ESP-EDU v1.1 posee varios pines están "comprometidos" con circuitería :

- **LEDs y Botones:** En ESP-EDU están comprometidos como se señala en el cuadro comparativo. Para mi ESP32 tengo montado en protoboard el mismo circuito para botones y LEDs que en ESP-EDU.

  Ver esquemáticos de mi ESP32 y ESP-EDU al final.

- **Sensores de 5V:** El ESP-EDU posee un chip TXB0104PW. Este integrado adapta los voltajes de 3.3V del ESP32-C6 a 5V para un puerto específico (ideal para el sensor ultrasónico HC-SR04 en pines GPIO2 y GPIO3). Mi ESP32 no posee esto; pero se montó un divisor de tensión para captar ECHO, tal como el esquemático de ESP-EDU, y el mismo sensor se alimenta con los 5V de mi kit. Luego se agregaron diferentes circuitos para el proyecto (documentado en cada módulo correspondiente).

Análisis de la Salida por Consola y Configuración

A partir del log del monitor serie que se ejecutó para ambas placas, se extraen hallazgos técnicos cruciales. Las configuraciones de mi repositorio actual están limitando artificialmente el hardware.

```
W (200) spi_flash: Detected size(8192k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
```

Limitación de Memoria Flash (Afecta a ambas): Este *Warning* es el más importante de todo el log. Inidica que el hardware físico de ambas placas (tanto la mía como la de la facultad) detectó un chip de memoria

Flash de **8 MB (8192 KB)**. Sin embargo, el archivo sdkconfig de mi proyecto (configuración del ESP-IDF) está indicando al compilador que genere una tabla de particiones para solo **2 MB (2048 KB)**. El sistema obedece al software y desperdicia 6 MB de espacio. Para aprovechar el hardware, se puede modificar el archivo sdkconfig pero se prefirió mantener en 1 MB de espacio para que este límite oficie de condición para el proyecto. Dentro de la carpeta del proyecto se encuentran las versiones de

sdkconfig y partitions.csv, correspondiente a este proyecto pero ampliado a otras funciones que escapan al objetivo de este proyecto (acceso por mDNS local, y DNS público, para visualización en tiempo real de variables fisiológicas, batería de respaldo, entre otros).

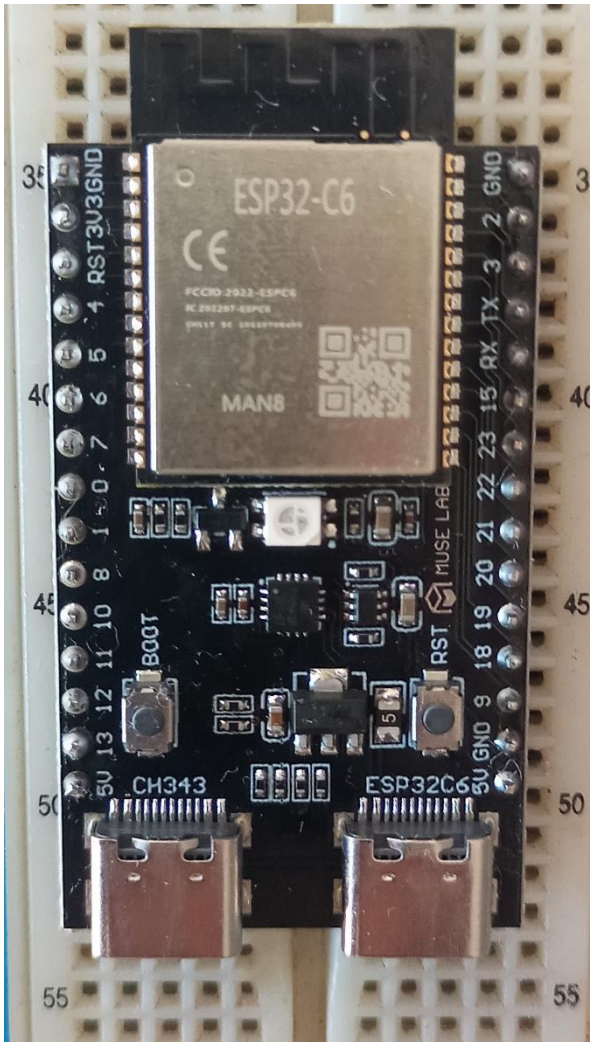
```
I (24) boot: chip revision: v0.2(Muse Lab) I (24) boot:chip revision: v0.0(ESP-EDU)
```

Revisión del Silicio: mi kit Muse Lab posee un chip de manufactura más reciente (v0.2) en comparación con el lote más antiguo (v0.0) soldado en el ESP-EDU. Las revisiones de silicio corrigen pequeños errores

("erratas") internos a nivel de registros de bajo consumo o temporizaciones SPI. A nivel de mi desarrollo en C utilizando el framework de alto nivel ESP-IDF, **esto no representa ninguna diferencia**, ya que el framework aplica parches de software automáticos en el arranque dependiendo de la versión detectada.

```
Cantidad de nucleos: 1 Memoria Heap Libre Actual: 475008 bytes
```

Arquitectura y Memoria Dinámica (RAM): El log comprueba reporta correctamente 1 núcleo de procesamiento. Además, se observa que, tras iniciar FreeRTOS, el stack de red y los servicios base, te quedan aproximadamente **475 KB de RAM (Heap) libres** para las variables, tareas, y almacenamiento de estructuras dinámicas. Esta cifra es completamente idéntica y consistente en ambas placas.





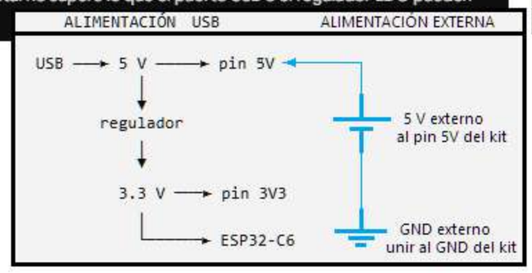
COMPARATIVA pines ESP32 Facultad vs. mi ESP32

ESP32 Facultad
ESP-EDU v1.1
Catedra de Electrónica Programable
Febrero 2024 ver 1 rev 0
Facultad de Ingeniería UNER

Respecto al origen y la distribución de energía en tu kit de desarrollo:

- **El pin de 5V (a veces marcado como VBUS):** Este voltaje no lo genera la placa. Proviene directamente del cable USB que conectas. Pasa por un diodo o fusible de protección superficial, pero esencialmente, cuando conectas algo a este pin, estás extrayendo corriente directa de la fuente original (el puerto de tu PC o cargador).
- **El pin de 3.3V:** Este voltaje lo genera un componente de montaje superficial soldado en la placa llamado **regulador de voltaje LDO** (Low Dropout). Este regulador toma los 5V brutos que entran por el USB y los reduce de manera lineal y estable a 3.3V.
- **El sentido del flujo:** La corriente ingresa por el puerto USB. El flujo se divide internamente: una parte queda disponible en los pines de 5V para que puedas alimentar componentes externos (como el HC-SR04). El resto ingresa al regulador LDO. La salida de 3.3V del regulador fluye internamente para alimentar el procesador ESP32-C6 y la memoria flash, y también se dirige hacia los pines marcados como "3.3V" de la placa.

En resumen, los pines de voltaje del kit de desarrollo actúan como "fuentes de salida" hacia tu protoboard. El sentido de la corriente va desde la placa hacia tus componentes externos, siempre y cuando el consumo total no supere lo que el puerto USB o el regulador LDO pueden entregar.







Clasificación de Pines (MuseLab nanoESP32-C6)

Pines de uso libre (Sin restricciones)

Puedes usarlos sin problemas tanto para entradas como para salidas:

- **GPIO 0, 1, 2, 3, 4, 5, 6, 7:** Pines estándar de E/S. Los pines 0 al 6 también funcionan como entradas analógicas (ADC1).
- **GPIO 10, 11, 18, 19, 20, 21, 22, 23:** Libres para cualquier uso digital, SPI, I2C o PWM.

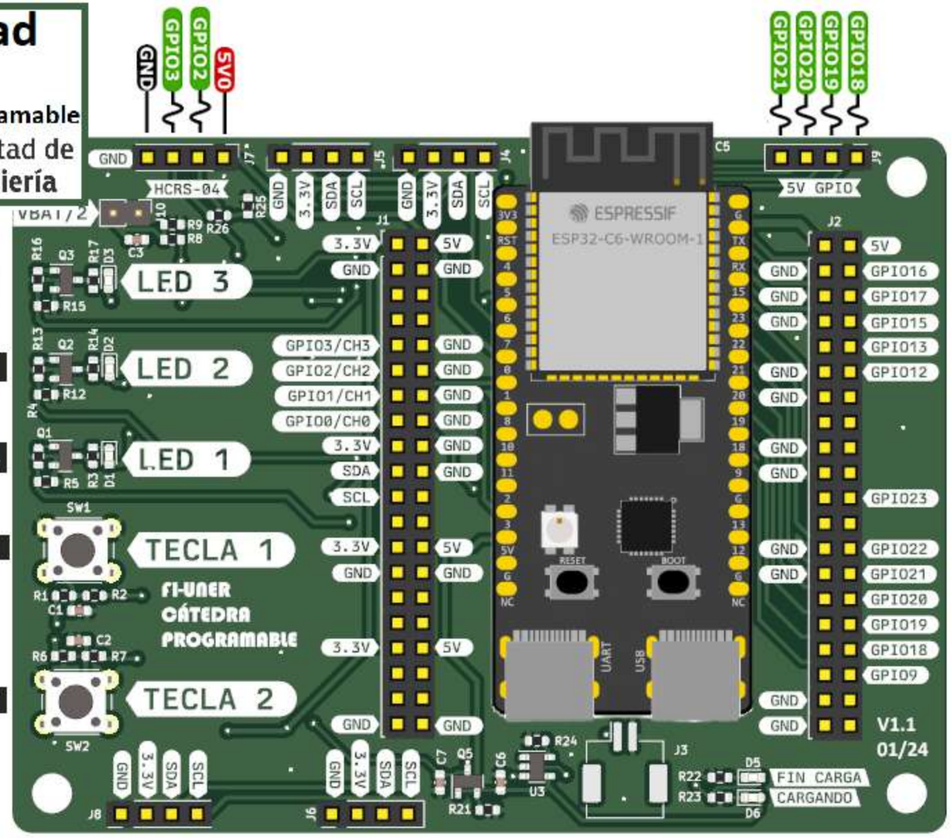
Pines con cuidado (Strapping, USB y UART)

- St **GPIO 9:** Pin de Boot principal (vía botón BOOT). Debe recibir un nivel **HIGH** durante el arranque. Si usas entradas con pulsadores, usa siempre resistencias de Pull-Up a 3.3V.

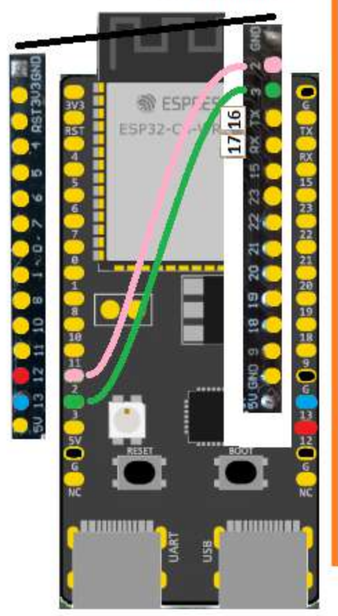
  Efecto: Pone el **GPIO 9 en nivel bajo (LOW / 0V).** Este es el pin de *Bootloader* (arranque). Si lo mantienes presionado mientras enciendes o reinicias la placa, fuerzas al chip a entrar en **modo descarga / flasheo de firmware.**
- St **GPIO 8:** Pin de Strapping secundario. Tiene un Pull-Up interno en el chip; evita forzarlo a GND con resistencias externas de bajo valor durante el encendido.
- St **GPIO 15:** Pin de Strapping (relacionado con JTAG / fuente de boot). No conviene colocarle cargas pesadas a masa durante el arranque.
- **GPIO 12 (USB D-) y GPIO 13 (USB D+):** Están conectados internamente al puerto USB-C derecho (etiquetado como `ESP32C6` , que provee USB nativo y JTAG). Si utilizas ese puerto para comunicación o depuración, no conectes componentes a estos pines.
- **GPIO 16 (TX) y GPIO 17 (RX):** Conectados al chip convertidor `CH343` del puerto USB izquierdo. Si usas la consola por puerto serie o la carga por el puerto CH343, mantén estos pines desocupados. `U0TXD` `U0RXD`

Pines a evitar / Flash / Solo entrada

- **Pines de Flash interna:** La placa **no expone** en los cabezales los pines de la SPI Flash integrada (GPIO 24 al 30), previniendo interferencias en el bus de memoria.
- **Pines de solo entrada: No existen en el ESP32-C6.** A diferencia del ESP32 clásico (que tenía los pines GPI 34 al 39 exclusivamente como entradas), en el ESP32-C6 **todos** los GPIOs expuestos en la placa son bidireccionales y pueden configurarse tanto de entrada como de salida.



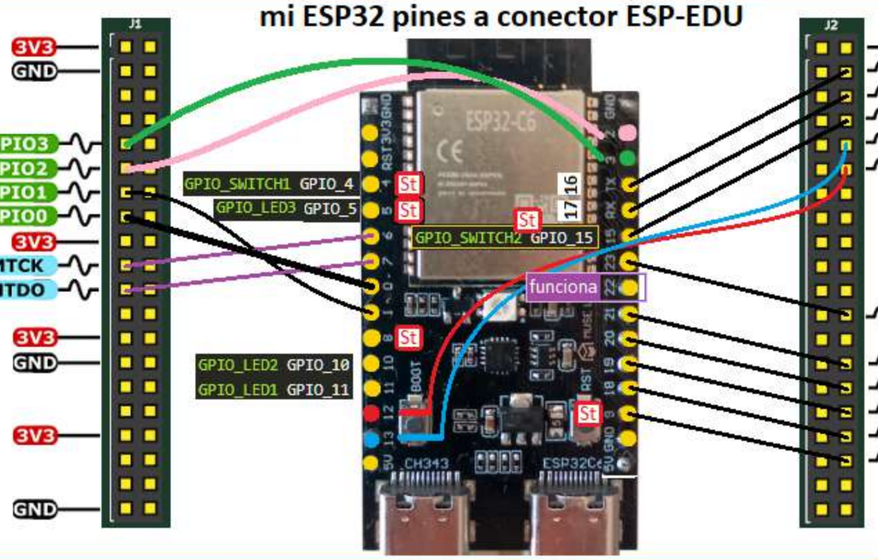
desfase de un pin más arriba para que coincida con la ESP-EDU

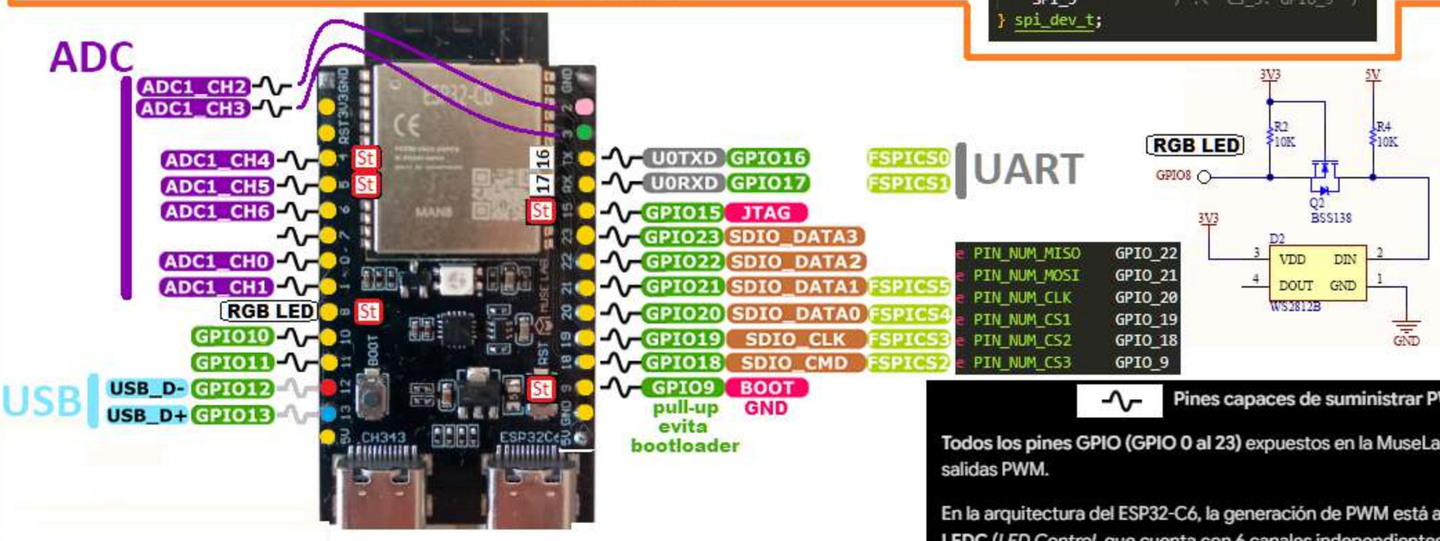


mi ESP32



mi ESP32 pines a conector ESP-EDU





El GPIO_8 puede usarse como entrada de sensores que normalmente está en algo.

El LED WS2812B (D2) no se encenderá accidentalmente porque exige una trama digital de 800 kHz con tiempos de nanosegundos para responder.

Table 2-11. Default Configuration of Strapping Pins

Strapping Pin		Default Configuration		Bit Value
MTMS	St	GPIO4	Floating	–
MTDI	St	GPIO5	Floating	–
GPIO8	St		Floating	PULL-UP = FLASHEA, PULL-DOWN = NO FLASHEA
GPIO9	St		Pull-up	1
GPIO15	St		Floating	–

ESP32-C6 Series Datasheet v0.5 **pág 26**

En la página 26 del datasheet (sección 2.6 Strapping Pins), se especifica que el control del muestreo y borde de reloj de SDIO (SDIO sampling and driving clock edge) se determina mediante los pines MTMS y MTDI

- **MTMS corresponde a GPIO4.**
- **MTDI corresponde a GPIO5.**

Evita forzar tensiones durante el arranque: No conectes circuitos externos que fuercen a GPIO4 o GPIO5 a un nivel lógico alto (3.3V) o bajo (GND) mediante resistencias de pull-up o pull-down fuertes justo en el momento en que se aplica energía o se presiona el botón de RESET.

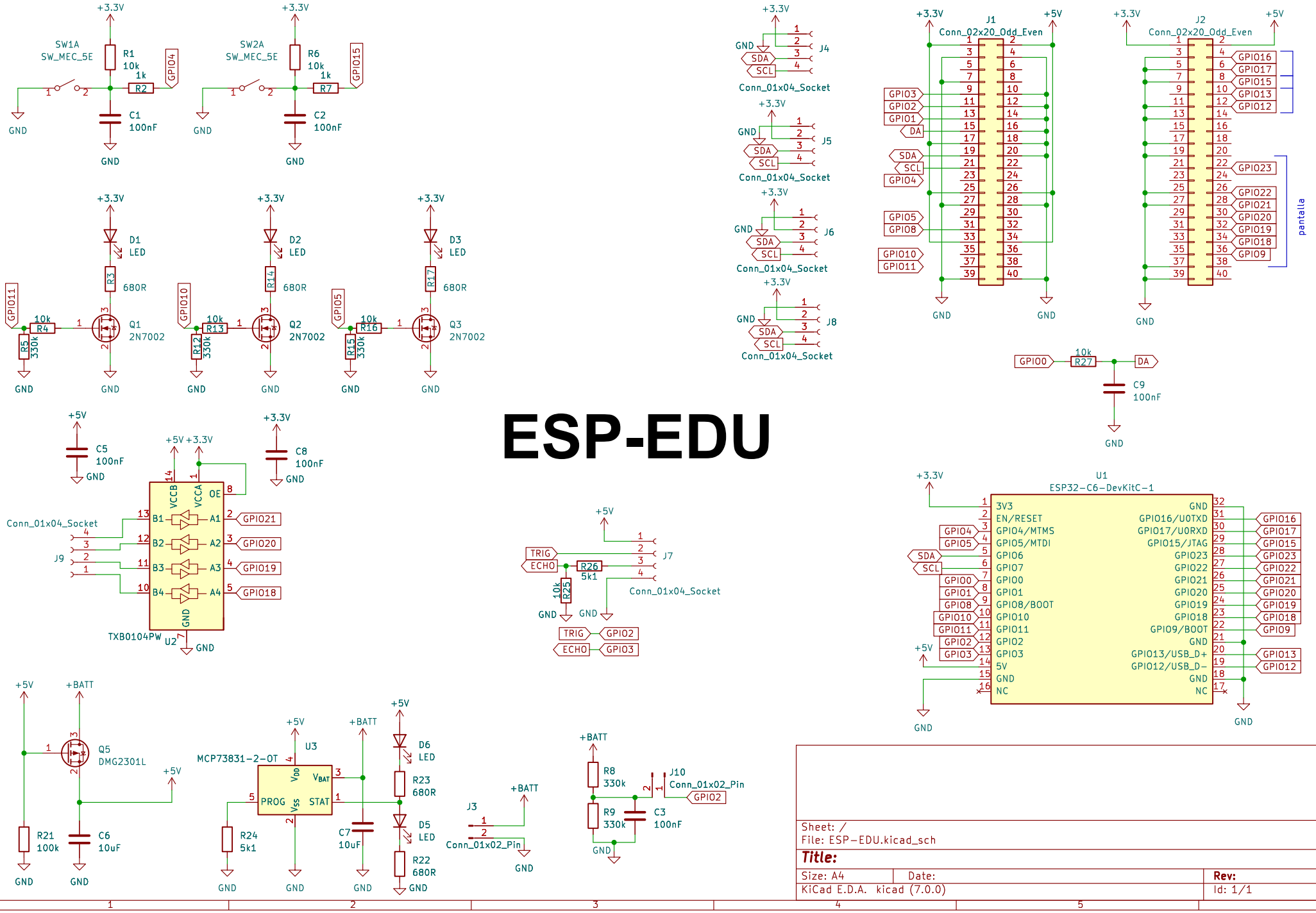
inmunidad (fuerte) 1K < R < 10K (débil) inmunidad

Pines capaces de suministrar PWM

Todos los pines GPIO (GPIO 0 al 23) expuestos en la MuseLab nanoESP32-C6 pueden generar salidas PWM.

En la arquitectura del ESP32-C6, la generación de PWM está a cargo de los periféricos internos **LEDC** (*LED Control*, que cuenta con 6 canales independientes) y **MCPWM** (*Motor Control PWM*). Gracias a la matriz interna de pines (**GPIO Matrix**) desarrollada por Espressif, cualquier canal de estos módulos se puede enrocar o asignar por software a **cualquiera** de las salidas digitales de la placa.

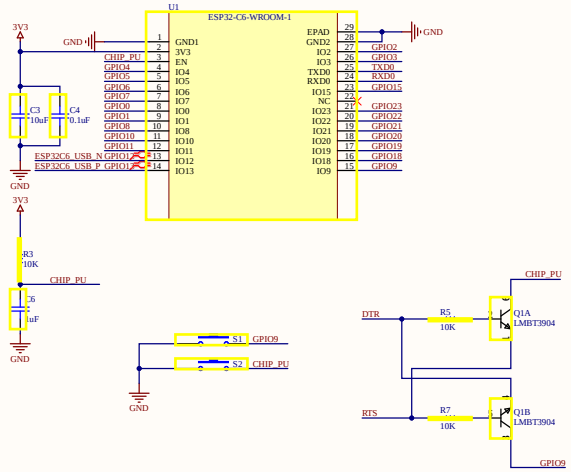
- **Pines ideales para PWM:**
  `GPIO 0, 1, 2, 3, 4, 5, 6, 10, 11, 18, 19, 20, 21, 22 y 23`
- **Pines con precaución para PWM:** Evita usar los pines de *strapping* (`GPIO 9` u `8`) si la carga (como la base de un transistor o un driver) impone un estado lógico alto/bajo que altere el arranque antes de que el código tome el control.


ESP-EDU
SW1A
SW_MEC_5E
R1
10k
1k
R2
C1
100nF
GPIO4
SW2A
SW_MEC_5E
R6
10k
1k
R7
C2
100nF
GPIO15
D1
LED
D2
LED
D3
LED
680R
R3
R14
R17
Q1
2N7002
Q2
2N7002
Q3
2N7002
R4
R13
R16
R5
R12
R15
330k
GPIO11
GPIO10
GPIO5
J4
J5
J6
J8
Conn_01x04_Socket
SDA
SCL
J1
J2
Conn_02x20_Odd_Even
pantalla
GPIO0
R27
DA
C9
100nF
C5
100nF
C8
100nF
J9
TXB0104PW
U2
VCCB
VCCA
OE
GPIO21
GPIO20
GPIO19
GPIO18
J7
TRIG
ECHO
R26
5k1
R25
10k
GPIO2
GPIO3
U1
ESP32-C6-DevKitC-1
Q5
DMG2301L
R21
100k
C6
10uF
U3
MCP73831-2-OT
R24
5k1
C7
10uF
D6
LED
D5
LED
R23
680R
R22
680R
J3
Conn_01x02_Pin
R8
330k
R9
330k
C3
100nF
J10
Sheet: /
File: ESP-EDU.kicad_sch
Title:
Size: A4
Date:
Rev:
KiCad E.D.A. kicad (7.0.0)
Id: 1/1

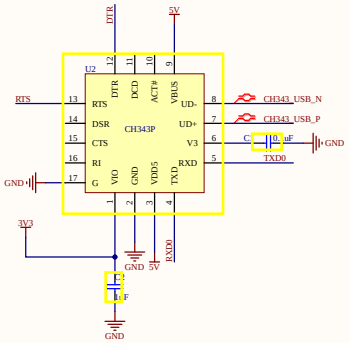
nanoESP32-C6 v1.0

mi ESP32-C6

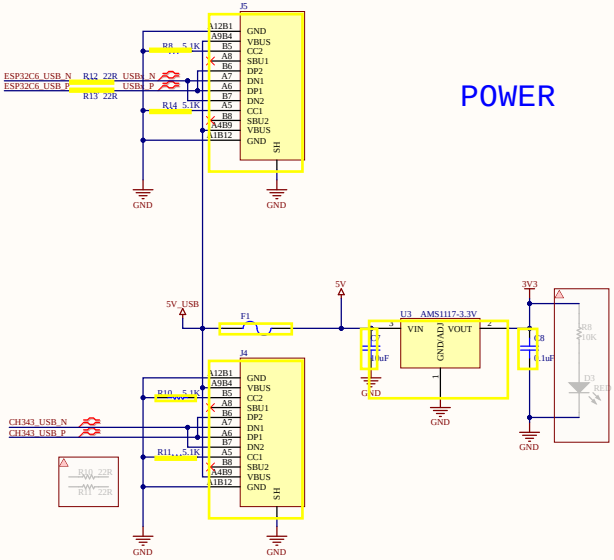
ESP32-C6-WROOM-1



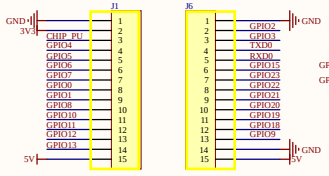
USB-to-Serial



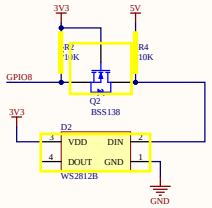
POWER



HEADER



RGB LED



Title		
Size: A2	Number	Revision
Date: 4/25/2023		Sheet of
File: F:\private\...\nanoESP32C6.SchDoc		Drawn By: